Practical 10:

Aim: Distance Vector Routing Algorithm in C

Distance Vector Routing Algorithm in **C** — often used in network simulations to demonstrate how routers share distance vectors to calculate the shortest paths.

Algorithm: Distance Vector Routing

Input:

- Number of nodes in the network, `N`
- Cost matrix `C[N][N]` where `C[i][j]` is the cost to go from node `i` to node `j`

Steps:

1. For each node `i` ($1 \leq i \leq N$):
    - Initialize its distance vector `DV[i]`:
        - `DV[i][j] = C[i][j]` for all `j`
    - Set the next hop to `j` if there is a direct connection; otherwise, to -1
2. Repeat the following steps until no distance vector changes:
    - For each node `i`:
        - For each destination node `j`:
            - For each neighbor `k` of node `i`:
                - If `DV[i][j] > C[i][k] + DV[k][j]`:
                    - Update `DV[i][j] = C[i][k] + DV[k][j]`
                    - Set next hop to `k`
3. After convergence, print the final distance vectors (routing tables) for all nodes

Explanation:

- Each node exchanges its distance vector with its immediate neighbors.
- On receiving a neighbor's vector, it updates its own vector if it finds a shorter path.
- This repeats until no update is needed (network converges).

C program

```
#include <stdio.h>

#define MAX_NODES 10
```

```
#define INF 9999

struct Node {
    int dist[MAX_NODES];
    int from[MAX_NODES];
} rt[MAX_NODES];

int main() {
    int cost[MAX_NODES][MAX_NODES], nodes, i, j, k, updated;

    printf("Enter number of nodes: ");
    scanf("%d", &nodes);

    printf("Enter cost matrix (use 999 for ∞/no connection):\n");
    for (i = 0; i < nodes; i++) {
        for (j = 0; j < nodes; j++) {
            scanf("%d", &cost[i][j]);
            cost[i][i] = 0;
            rt[i].dist[j] = cost[i][j];
            rt[i].from[j] = j;
        }
    }

    do {
        updated = 0;
        for (i = 0; i < nodes; i++) {
            for (j = 0; j < nodes; j++) {
                for (k = 0; k < nodes; k++) {
                    if (rt[i].dist[j] > cost[i][k] + rt[k].dist[j]) {
                        rt[i].dist[j] = cost[i][k] + rt[k].dist[j];
                        rt[i].from[j] = k;
                        updated = 1;
                    }
                }
            }
        }
    } while (updated);

    // Display routing table
    for (i = 0; i < nodes; i++) {
        printf("\nRouting table for Node %d:\n", i);
        printf("Destination\tNext Hop\tDistance\n");
        for (j = 0; j < nodes; j++) {
            printf("%d\t\t%d\t\t%d\n", j, rt[i].from[j], rt[i].dist[j]);
        }
    }

    return 0;
}
```

Sample Input

```
Enter number of nodes: 3
Enter cost matrix:
0 2 7
```

```
2 0 1
7 1 0
```

Compile and Run

```
gcc distance_vector.c -o dv
./dv
```

Explanation

- Each router (node) maintains a table with:
  - Distance to every other node
  - Next hop to reach each node
- The algorithm loops until no updates are made, ensuring the shortest paths are computed.

Output:

```
sit-lab-1@sit-lab-1-OptiPlex-3280-AIO:~$ gedit p10.c
sit-lab-1@sit-lab-1-OptiPlex-3280-AIO:~$ gcc p10.c -o p10
sit-lab-1@sit-lab-1-OptiPlex-3280-AIO:~$ ./p10
Enter number of nodes: 3
Enter cost matrix (use 999 for infinity / no connection):
3 5 1
6 8 2
0 2 1

Routing table for Node 0:
Destination     Next Hop        Distance
0               0               0
1               2               3
2               2               1

Routing table for Node 1:
Destination     Next Hop        Distance
0               2               2
1               1               0
2               2               2

Routing table for Node 2:
Destination     Next Hop        Distance
0               0               0
1               1               2
2               2               0
```